HDANI BACKEND

Placement Test System

API Documentation — 7 July 2026

Contents

1. Placement Flow (Main Exam)
2. Placement Start & Session
3. Deliver Exam (GET)
4. Submit Section (POST)
5. Final Placement Decision
6. Speaking AI Evaluation
7. Writing AI Evaluation
8. Audio / TTS Endpoints
9. Configuration
10. How to Run

1. Placement Flow

The placement test consists of 4 mandatory sections in strict order:

- Reading !’ Listening !’ Writing !’ Speaking

Each section must be completed before the next is unlocked. After Speaking is submitted, the backend calculates the final placement decision.

2. Start Placement Session

POST /api/placement/progression/start

Creates a new placement session. Returns sessionId and the first section (reading).

Request Body

```
{
  "estimatedLevel": "A1"
}
```

Response (200)

```
{
  "sessionId": "uuid",
  "level": "A1",
  "nextSection": "reading"
}
```

3. Deliver Exam

GET /api/placement/:level/:section

Fetches exam questions for the given level and section.

Parameters

- level: A1, A2, B1, B2, C1, C2
- section: reading, listening, writing, speaking

Reading Response (200) — 3 passages × 3 questions

```
{
  "examId": "reading_001",
  "sectionTitle": "Reading",
  "passages": [
    {
      "id": "p1",
      "title": "...",
      "text": "...",
      "questions": [
        { "id": "q1", "question": "...", "options": [...], "correctAnswer": 1 }
      ]
    }
  ]
}
```

Listening Response (200) — 3 random questions with audioUrl

```
{
  "examId": "listening_001",
  "sectionTitle": "Listening",
  "questions": [
    { "questionId": "q1", "audioUrl": "https://...", "question": "...", "options":
[...] }
```

```
  ]
}
```

Writing / Speaking Response (200)

```
{
  "examId": "writing_001",
  "title": "...",
  "instruction": "...",
  "prompts": ["What is your name?", "..."]
}
```

4. Submit Section

POST /api/placement/:level/:section/submit

Submits answers for a section. Sections 1–3 return a continue response. Section 4 (speaking) returns the final placement decision.

Reading Submit Body

```
{
  "sessionId": "uuid",
  "examId": "reading_001",
  "answers": [
    0,
    1,
    2,
    1,
    0,
    2,
    2,
    1,
    0
  ]
}
```

Listening Submit Body

```
{
  "sessionId": "uuid",
  "examId": "listening_001",
  "answers": [
    0,
    1,
    2
  ],
  "questionIds": [
    "q1",
    "q2",
    "q3"
  ]
}
```

Writing Submit Body

```
{
  "sessionId": "uuid",
  "examId": "writing_001",
  "text": "My name is test.",
  "percentage": 70
}
```

Speaking Submit Body

```
{
  "sessionId": "uuid",
```

```
    "examId": "speaking_001",
    "conversation": "Q: What is your name? A: Test.",
    "percentage": 65
  }
```

Continue Response (Sections 1–3)

```
  {
    "status": "continue",
    "nextSection": "listening"
  }
```

Final Response (Speaking — Section 4)

```
  {
    "success": true,
    "average": 67,
    "reading": 100,
    "listening": 33.33,
    "writing": 70,
    "speaking": 65,
    "decision": "STAY",
    "placementLevel": "A1"
  }
```

---

5. Final Placement Decision

Calculated automatically after Speaking is submitted. Logic lives in services/placementDecisionService.js.

Thresholds are read from utils/placementConfig.js:

- PASS_THRESHOLD = 75
- STAY_THRESHOLD = 45

Decision Rules

- average >= 75 !’ NEXT_LEVEL (advance to next CEFR level)
- 45 <= average < 75 !’ STAY (remain at current level)
- average < 45 !’ LOWER_LEVEL (move to previous level)
- A1 with average < 45 !’ STAY (no lower level)
- C2 with average >= 75 !’ STAY (max level reached)

NEXT_LEVEL Response

```
  {
    "success": true,
    "average": 76,
    "reading": 80,
    "listening": 72,
    "writing": 68,
    "speaking": 84,
    "decision": "NEXT_LEVEL",
    "currentLevel": "A1",
    "nextLevel": "A2"
  }
```

STAY Response

```
  {
    "success": true,
    "average": 60,
    "reading": 65,
    "listening": 55,
    "writing": 50,
    "speaking": 70,
    "decision": "STAY",
    "placementLevel": "A1"
```

```
}
```

LOWER_LEVEL Response

```
{
  "success": true,
  "average": 37,
  "reading": 30,
  "listening": 42,
  "writing": 35,
  "speaking": 41,
  "decision": "LOWER_LEVEL",
  "placementLevel": "A2"
}
```

---

6. Speaking AI Evaluation

POST /api/speaking/evaluate

Evaluates speaking using OpenRouter AI. Returns structured CEFR scores.

Request Body

```
{
  "level": "A1",
  "conversation": [
    {
      "question": "What is your name?",
      "answer": "My name is Ahmed."
    },
    {
      "question": "How old are you?",
      "answer": "I am 20 years old."
    }
  ]
}
```

Response (200)

```
{
  "success": true,
  "result": {
    "grammar": 18,
    "vocabulary": 15,
    "taskCompletion": 20,
    "fluency": 16,
    "total": 69,
    "feedback": "...",
    "strengths": ["..."],
    "weaknesses": ["..."],
    "cefrLevel": "A2"
  }
}
```

---

7. Writing AI Evaluation

POST /api/gemini/evaluate-writing

Evaluates writing using OpenRouter AI. Returns structured CEFR scores.

Request Body

```
{
  "level": "A1",
```

```
    "question": "Describe your family.",
    "answer": "I have a small family."
  }
```

Response (200)

```
{
  "grammar": 18,
  "vocabulary": 16,
  "taskCompletion": 20,
  "coherence": 17,
  "total": 71,
  "feedback": "...",
  "strengths": ["..."],
  "weaknesses": ["..."],
  "cefrLevel": "A2"
}
```

---

8. Audio / TTS Endpoints

POST /api/audio/generate

Generates speech from text using Edge TTS, uploads to Cloudinary, returns URL.

Request Body

```
{
  "text": "Hello world",
  "voice": "teacher"
}
```

Response (200)

```
{ "audioUrl": "https://res.cloudinary.com/..." }
```

GET /test/audio

Simple test endpoint. Returns a static audio file list.

POST /api/gemini/evaluate-speaking

Alternative speaking evaluation endpoint using OpenRouter AI.

---

9. Configuration

utils/placementConfig.js

```
LEVEL_ORDER = ["A1", "A2", "B1", "B2", "C1", "C2"]
PASS_THRESHOLD = 75
STAY_THRESHOLD = 45
```

Environment Variables (.env)

```
PORT=3000
MONGODB_URI=...
CLOUDINARY_CLOUD_NAME=...
CLOUDINARY_API_KEY=...
CLOUDINARY_API_SECRET=...
OPENROUTER_API_KEY=...
```

---

10. How to Run

Start Server

```
node server.js
```

Server starts on http://localhost:3000 by default.

Frontend Pages

- http://localhost:3000 — Main placement test UI
- http://localhost:3000/tts-test.html — TTS audio test
- http://localhost:3000/speaking-test/ — Speaking AI evaluation test

Offline Audio Generation

```
node scripts/generateListeningAudio.js
```

Pre-generates all listening audio URLs via Edge TTS and stores in Cloudinary.

Key Files

- services/placementDecisionService.js — New placement decision logic
- services/placementService.js — Main placement orchestration
- services/placementSessionService.js — In-memory session management
- services/placementProgressionService.js — Legacy progression logic
- services/elevenLabsService.js — Edge TTS implementation
- services/storageService.js — Cloudinary upload
- services/aiService.js — Speaking AI evaluation prompt
- services/speakingService.js — Speaking evaluation orchestration
- providers/openRouterProvider.js — AI provider (6-model fallback)
- utils/fileHelpers.js — Score calculation, flatten helpers
- scripts/generateListeningAudio.js — Offline audio pre-generation